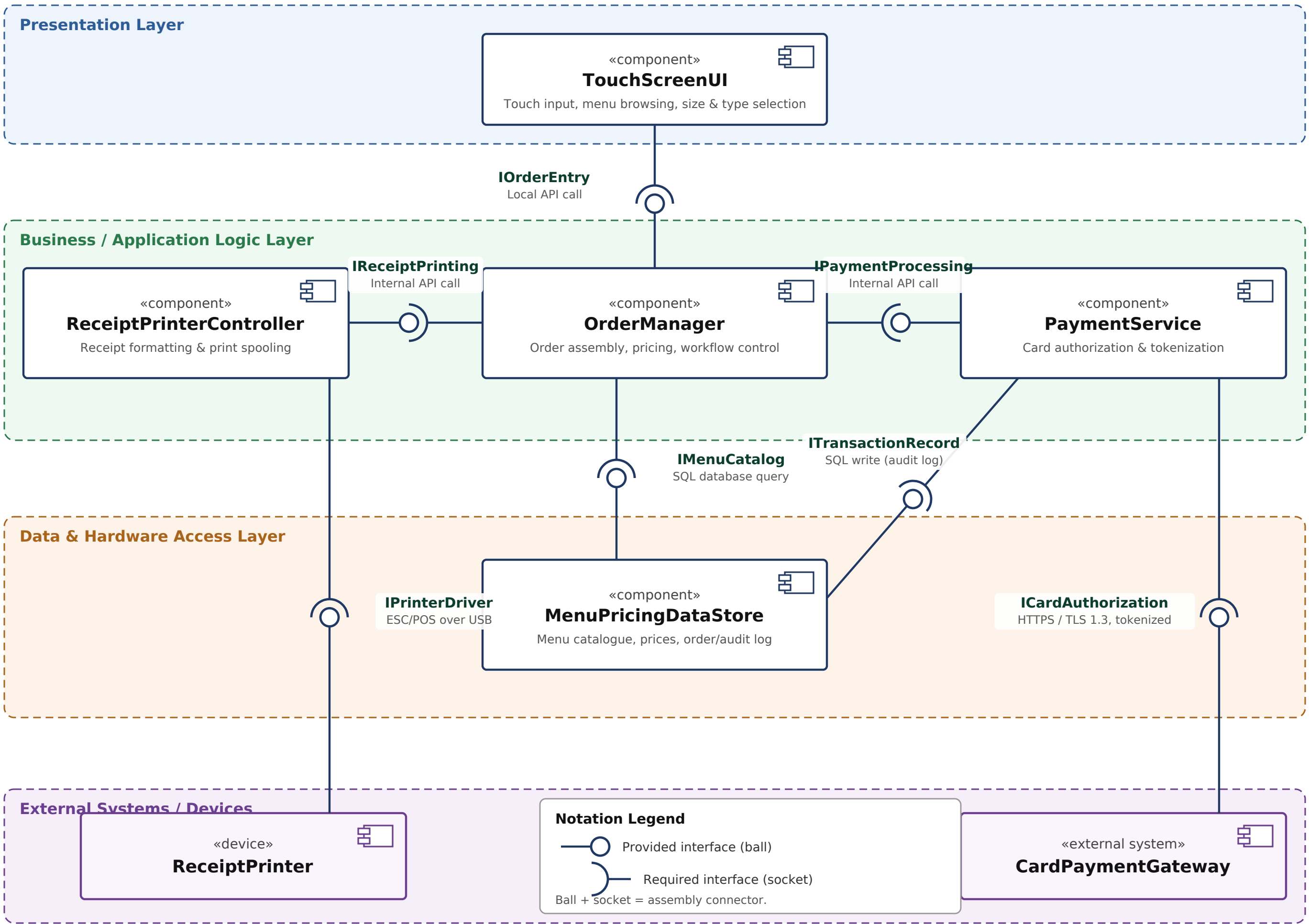


UML Component Diagram — Self-Service Coffee Kiosk System

Architectural Style: Layered (3-Tier) Architecture

Rohan Agarwal · PES1UG24AM921 · SE Lab 3



SE Lab 3 — Component Modelling & Architectural Pattern Selection

Architectural Justification: Self-Service Coffee Kiosk System

Rohan Agarwal | SRN: PES1UG24AM921 | Dept. of CSE, PES University

1. Architecture Selection

We chose the **Layered (3-Tier) Architecture** for the Self-Service Coffee Kiosk System. The system is organised into a Presentation layer (TouchScreenUI), a Business/Application Logic layer (OrderManager, PaymentService, ReceiptPrinterController) and a Data & Hardware Access layer (MenuPricingDataStore, printer driver), with all communication flowing through explicitly declared provided/required interfaces.

2. Justification — Two Scenario-Specific Reasons

Reason 1 — The problem domain is small, fixed and single-node. The kiosk offers exactly three coffee types, two sizes and one payment method, and the whole system runs on a single machine inside one café. There is no part of this workload that needs to scale independently, so the operational cost of Microservices (service discovery, inter-process network hops, distributed transactions across order and payment data) buys nothing here. A layered design gives the same separation of concerns through in-process calls, and keeps the deployable unit to one installable kiosk application.

Reason 2 — Hardware volatility must be isolated from business rules. The kiosk is bound to physical devices: a touch screen and a receipt printer. By confining device-specific code to ReceiptPrinterController and the hardware-access layer, a change of printer model (a different ESC/POS variant, say) is absorbed by one component — OrderManager continues to call the IReceiptPrinting interface unchanged. The same isolation applies to menu and pricing data: prices change frequently, but only MenuPricingDataStore behind IMenuCatalog is affected, never the UI or the ordering logic.

3. Security Advantage

Layering enforces a **single controlled path to sensitive data**. The TouchScreenUI has no direct access to the data store or to card data — it can only invoke IOrderEntry, so a compromise of the public-facing touch interface cannot read or modify pricing records or transaction logs. Card details never enter the presentation or data layers at all: PaymentService sends them straight to the CardPaymentGateway over TLS 1.3 and stores only the returned token, so no primary account number is ever written to MenuPricingDataStore. This confines PCI-DSS scope to one component instead of the whole system, and the layer boundaries themselves become natural, auditable choke points for input validation and for transaction logging via ITransactionRecord.

4. Performance Benefit

Every call on the ordering critical path — IOrderEntry, IMenuCatalog and IReceiptPrinting — is an **in-process call against local storage**, with no network round trip and no serialisation overhead. Menu rendering, size selection and price recalculation therefore respond to touch in a few milliseconds, which is what matters when customers are queueing in a busy café. The only remote interaction is ICardAuthorization to the payment gateway, and because the receipt can be formatted by ReceiptPrinterController while that authorisation is in flight, the perceived wait at the kiosk stays close to the gateway's own latency.

5. Trade-off Accepted

The main cost of this choice is scalability across sites: if the café later runs a fleet of kiosks with centralised menu management and analytics, the data layer becomes a bottleneck and a shared catalogue service would need to be extracted. Given the stated single-kiosk scope, that cost is acceptable, and the clean interface boundaries make such an extraction straightforward later.